

$$n = n_0 + n_1 + \cdots + n_{m-1} , \quad (11.1)$$

and the expected value of  $n_j$  is  $E[n_j] = \alpha = n/m$ .

We assume that  $O(1)$  time suffices to compute the hash value  $h(k)$ , so that the time required to search for an element with key  $k$  depends linearly on the length  $n_{h(k)}$  of the list  $T[h(k)]$ . Setting aside the  $O(1)$  time required to compute the hash function and to access slot  $h(k)$ , we'll consider the expected number of elements examined by the search algorithm, that is, the number of elements in the list  $T[h(k)]$  that the algorithm checks to see whether any have a key equal to  $k$ . We consider two cases. In the first, the search is unsuccessful: no element in the table has key  $k$ . In the second, the search successfully finds an element with key  $k$ .

**Theorem 11.1**

In a hash table in which collisions are resolved by chaining, an unsuccessful search takes  $\Theta(1 + \alpha)$  time on average, under the assumption of independent uniform hashing.

**Proof** Under the assumption of independent uniform hashing, any key  $k$  not already stored in the table is equally likely to hash to any of the  $m$  slots. The expected time to search unsuccessfully for a key  $k$  is the expected time to search to the end of list  $T[h(k)]$ , which has expected length  $E[n_{h(k)}] = \alpha$ . Thus, the expected number of elements examined in an unsuccessful search is  $\alpha$ , and the total time required (including the time for computing  $h(k)$ ) is  $\Theta(1 + \alpha)$ . ■

The situation for a successful search is slightly different. An unsuccessful search is equally likely to go to any slot of the hash table. A successful search, however, cannot go to an empty slot, since it is for an element that is present in one of the linked lists. We assume that the element searched for is equally likely to be any one of the elements in the table, so the longer the list, the more likely that the search is for one of its elements. Even so, the expected search time still turns out to be  $\Theta(1 + \alpha)$ .

**Theorem 11.2**

In a hash table in which collisions are resolved by chaining, a successful search takes  $\Theta(1 + \alpha)$  time on average, under the assumption of independent uniform hashing.

**Proof** We assume that the element being searched for is equally likely to be any of the  $n$  elements stored in the table. The number of elements examined during a successful search for an element  $x$  is 1 more than the number of elements that appear before  $x$  in  $x$ 's list. Because new elements are placed at the front of the list,

elements before  $x$  in the list were all inserted after  $x$  was inserted. Let  $x_i$  denote the  $i$ th element inserted into the table, for  $i = 1, 2, \dots, n$ , and let  $k_i = x_i.\text{key}$ .

Our analysis uses indicator random variables extensively. For each slot  $q$  in the table and for each pair of distinct keys  $k_i$  and  $k_j$ , we define the indicator random variable

$$X_{ijq} = \mathbf{I} \{ \text{the search is for } x_i, h(k_i) = q, \text{ and } h(k_j) = q \} .$$

That is,  $X_{ijq} = 1$  when keys  $k_i$  and  $k_j$  collide at slot  $q$  and the search is for element  $x_i$ . Because  $\Pr \{ \text{the search is for } x_i \} = 1/n$ ,  $\Pr \{ h(k_i) = q \} = 1/m$ ,  $\Pr \{ h(k_j) = q \} = 1/m$ , and these events are all independent, we have that  $\Pr \{ X_{ijq} = 1 \} = 1/nm^2$ . Lemma 5.1 on page 130 gives  $E[X_{ijq}] = 1/nm^2$ .

Next, we define, for each element  $x_j$ , the indicator random variable

$$\begin{aligned} Y_j &= \mathbf{I} \{ x_j \text{ appears in a list prior to the element being searched for} \} \\ &= \sum_{q=0}^{m-1} \sum_{i=1}^{j-1} X_{ijq} , \end{aligned}$$

since at most one of the  $X_{ijq}$  equals 1, namely when the element  $x_i$  being searched for belongs to the same list as  $x_j$  (pointed to by slot  $q$ ), and  $i < j$  (so that  $x_i$  appears after  $x_j$  in the list).

Our final random variable is  $Z$ , which counts how many elements appear in the list prior to the element being searched for:

$$Z = \sum_{j=1}^n Y_j .$$

Because we must count the element being searched for as well as all those preceding it in its list, we wish to compute  $E[Z + 1]$ . Using linearity of expectation (equation (C.24) on page 1192), we have

$$\begin{aligned} E[Z + 1] &= E \left[ 1 + \sum_{j=1}^n Y_j \right] \\ &= 1 + E \left[ \sum_{j=1}^n \sum_{q=0}^{m-1} \sum_{i=1}^{j-1} X_{ijq} \right] \\ &= 1 + E \left[ \sum_{q=0}^{m-1} \sum_{j=1}^n \sum_{i=1}^{j-1} X_{ijq} \right] \\ &= 1 + \sum_{q=0}^{m-1} \sum_{j=1}^n \sum_{i=1}^{j-1} E[X_{ijq}] \quad (\text{by linearity of expectation}) \end{aligned}$$

$$\begin{aligned}
&= 1 + \sum_{q=0}^{m-1} \sum_{j=1}^n \sum_{i=1}^{j-1} \frac{1}{nm^2} \\
&= 1 + m \cdot \frac{n(n-1)}{2} \cdot \frac{1}{nm^2} \quad (\text{by equation (A.2) on page 1141}) \\
&= 1 + \frac{n-1}{2m} \\
&= 1 + \frac{n}{2m} - \frac{1}{2m} \\
&= 1 + \frac{\alpha}{2} - \frac{\alpha}{2n}.
\end{aligned}$$

Thus, the total time required for a successful search (including the time for computing the hash function) is  $\Theta(2 + \alpha/2 - \alpha/2n) = \Theta(1 + \alpha)$ . ■

What does this analysis mean? If the number of elements in the table is at most proportional to the number of hash-table slots, we have  $n = O(m)$  and, consequently,  $\alpha = n/m = O(m)/m = O(1)$ . Thus, searching takes constant time on average. Since insertion takes  $O(1)$  worst-case time and deletion takes  $O(1)$  worst-case time when the lists are doubly linked (assuming that the list element to be deleted is known, and not just its key), we can support all dictionary operations in  $O(1)$  time on average.

The analysis in the preceding two theorems depends only on two essential properties of independent uniform hashing: uniformity (each key is equally likely to hash to any one of the  $m$  slots), and independence (so any two distinct keys collide with probability  $1/m$ ).

## Exercises

### 11.2-1

You use a hash function  $h$  to hash  $n$  distinct keys into an array  $T$  of length  $m$ . Assuming independent uniform hashing, what is the expected number of collisions? More precisely, what is the expected cardinality of  $\{\{k_1, k_2\} : k_1 \neq k_2 \text{ and } h(k_1) = h(k_2)\}$ ?

### 11.2-2

Consider a hash table with 9 slots and the hash function  $h(k) = k \bmod 9$ . Demonstrate what happens upon inserting the keys 5, 28, 19, 15, 20, 33, 12, 17, 10 with collisions resolved by chaining.

**11.2-3**

Professor Marley hypothesizes that he can obtain substantial performance gains by modifying the chaining scheme to keep each list in sorted order. How does the professor's modification affect the running time for successful searches, unsuccessful searches, insertions, and deletions?

**11.2-4**

Suggest how to allocate and deallocate storage for elements within the hash table itself by creating a “free list”: a linked list of all the unused slots. Assume that one slot can store a flag and either one element plus a pointer or two pointers. All dictionary and free-list operations should run in  $O(1)$  expected time. Does the free list need to be doubly linked, or does a singly linked free list suffice?

**11.2-5**

You need to store a set of  $n$  keys in a hash table of size  $m$ . Show that if the keys are drawn from a universe  $U$  with  $|U| > (n - 1)m$ , then  $U$  has a subset of size  $n$  consisting of keys that all hash to the same slot, so that the worst-case searching time for hashing with chaining is  $\Theta(n)$ .

**11.2-6**

You have stored  $n$  keys in a hash table of size  $m$ , with collisions resolved by chaining, and you know the length of each chain, including the length  $L$  of the longest chain. Describe a procedure that selects a key uniformly at random from among the keys in the hash table and returns it in expected time  $O(L \cdot (1 + 1/\alpha))$ .

---

**11.3 Hash functions**

For hashing to work well, it needs a good hash function. Along with being efficiently computable, what properties does a good hash function have? How do you design good hash functions?

This section first attempts to answer these questions based on two ad hoc approaches for creating hash functions: hashing by division and hashing by multiplication. Although these methods work well for some sets of input keys, they are limited because they try to provide a single fixed hash function that works well on any data—an approach called *static hashing*.

We then see that provably good average-case performance for *any* data can be obtained by designing a suitable *family* of hash functions and choosing a hash function at random from this family at runtime, independent of the data to be hashed. The approach we examine is called random hashing. A particular kind of random

hashing, universal hashing, works well. As we saw with quicksort in Chapter 7, randomization is a powerful algorithmic design tool.

### What makes a good hash function?

A good hash function satisfies (approximately) the assumption of independent uniform hashing: each key is equally likely to hash to any of the  $m$  slots, independently of where any other keys have hashed to. What does “equally likely” mean here? If the hash function is fixed, any probabilities would have to be based on the probability distribution of the input keys.

Unfortunately, you typically have no way to check this condition, unless you happen to know the probability distribution from which the keys are drawn. Moreover, the keys might not be drawn independently.

Occasionally you might know the distribution. For example, if you know that the keys are random real numbers  $k$  independently and uniformly distributed in the range  $0 \leq k < 1$ , then the hash function

$$h(k) = \lfloor km \rfloor$$

satisfies the condition of independent uniform hashing.

A good static hashing approach derives the hash value in a way that you expect to be independent of any patterns that might exist in the data. For example, the “division method” (discussed in Section 11.3.1) computes the hash value as the remainder when the key is divided by a specified prime number. This method may give good results, if you (somehow) choose a prime number that is unrelated to any patterns in the distribution of keys.

Random hashing, described in Section 11.3.2, picks the hash function to be used at random from a suitable family of hashing functions. This approach removes any need to know anything about the probability distribution of the input keys, as the randomization necessary for good average-case behavior then comes from the (known) random process used to pick the hash function from the family of hash functions, rather than from the (unknown) process used to create the input keys. We recommend that you use random hashing.

### Keys are integers, vectors, or strings

In practice, a hash function is designed to handle keys that are one of the following two types:

- A short nonnegative integer that fits in a  $w$ -bit machine word. Typical values for  $w$  would be 32 or 64.

- A short vector of nonnegative integers, each of bounded size. For example, each element might be an 8-bit byte, in which case the vector is often called a (byte) string. The vector might be of variable length.

To begin, we assume that keys are short nonnegative integers. Handling vector keys is more complicated and discussed in Sections 11.3.5 and 11.5.2.

### 11.3.1 Static hashing

Static hashing uses a single, fixed hash function. The only randomization available is through the (usually unknown) distribution of input keys. This section discusses two standard approaches for static hashing: the division method and the multiplication method. Although static hashing is no longer recommended, the multiplication method also provides a good foundation for “nonstatic” hashing—better known as random hashing—where the hash function is chosen at random from a suitable family of hash functions.

#### The division method

The *division method* for creating hash functions maps a key  $k$  into one of  $m$  slots by taking the remainder of  $k$  divided by  $m$ . That is, the hash function is

$$h(k) = k \bmod m .$$

For example, if the hash table has size  $m = 12$  and the key is  $k = 100$ , then  $h(k) = 4$ . Since it requires only a single division operation, hashing by division is quite fast.

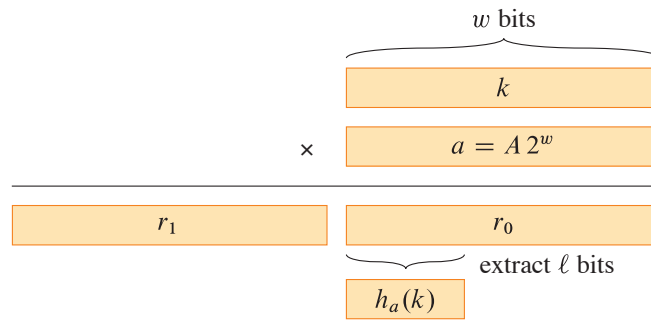
The division method may work well when  $m$  is a prime not too close to an exact power of 2. There is no guarantee that this method provides good average-case performance, however, and it may complicate applications since it constrains the size of the hash tables to be prime.

#### The multiplication method

The general *multiplication method* for creating hash functions operates in two steps. First, multiply the key  $k$  by a constant  $A$  in the range  $0 < A < 1$  and extract the fractional part of  $kA$ . Then, multiply this value by  $m$  and take the floor of the result. That is, the hash function is

$$h(k) = \lfloor m (kA \bmod 1) \rfloor ,$$

where “ $kA \bmod 1$ ” means the fractional part of  $kA$ , that is,  $kA - \lfloor kA \rfloor$ . The general multiplication method has the advantage that the value of  $m$  is not critical and you can choose it independently of how you choose the multiplicative constant  $A$ .



**Figure 11.4** The multiply-shift method to compute a hash function. The  $w$ -bit representation of the key  $k$  is multiplied by the  $w$ -bit value  $a = A \cdot 2^w$ . The  $\ell$  highest-order bits of the lower  $w$ -bit half of the product form the desired hash value  $h_a(k)$ .

### The multiply-shift method

In practice, the multiplication method is best in the special case where the number  $m$  of hash-table slots is an exact power of 2, so that  $m = 2^\ell$  for some integer  $\ell$ , where  $\ell \leq w$  and  $w$  is the number of bits in a machine word. If you choose a fixed  $w$ -bit positive integer  $a = A 2^w$ , where  $0 < A < 1$  as in the multiplication method so that  $a$  is in the range  $0 < a < 2^w$ , you can implement the function on most computers as follows. We assume that a key  $k$  fits into a single  $w$ -bit word.

Referring to Figure 11.4, first multiply  $k$  by the  $w$ -bit integer  $a$ . The result is a  $2w$ -bit value  $r_1 2^w + r_0$ , where  $r_1$  is the high-order  $w$ -bit word of the product and  $r_0$  is the low-order  $w$ -bit word of the product. The desired  $\ell$ -bit hash value consists of the  $\ell$  most significant bits of  $r_0$ . (Since  $r_1$  is ignored, the hash function can be implemented on a computer that produces only a  $w$ -bit product given two  $w$ -bit inputs, that is, where the multiplication operation computes modulo  $2^w$ .)

In other words, you define the hash function  $h = h_a$ , where

$$h_a(k) = (ka \bmod 2^w) \ggg (w - \ell) \quad (11.2)$$

for a fixed nonzero  $w$ -bit value  $a$ . Since the product  $ka$  of two  $w$ -bit words occupies  $2w$  bits, taking this product modulo  $2^w$  zeroes out the high-order  $w$  bits ( $r_1$ ), leaving only the low-order  $w$  bits ( $r_0$ ). The  $\ggg$  operator performs a logical right shift by  $w - \ell$  bits, shifting zeros into the vacated positions on the left, so that the  $\ell$  most significant bits of  $r_0$  move into the  $\ell$  rightmost positions. (It's the same as dividing by  $2^{w-\ell}$  and taking the floor of the result.) The resulting value equals the  $\ell$  most significant bits of  $r_0$ . The hash function  $h_a$  can be implemented with three machine instructions: multiplication, subtraction, and logical right shift.

As an example, suppose that  $k = 123456$ ,  $\ell = 14$ ,  $m = 2^{14} = 16384$ , and  $w = 32$ . Suppose further that we choose  $a = 2654435769$  (following a suggestion

of Knuth [261]). Then  $ka = 327706022297664 = (76300 \cdot 2^{32}) + 17612864$ , and so  $r_1 = 76300$  and  $r_0 = 17612864$ . The 14 most significant bits of  $r_0$  yield the value  $h_a(k) = 67$ .

Even though the multiply-shift method is fast, it doesn't provide any guarantee of good average-case performance. The universal hashing approach presented in the next section provides such a guarantee. A simple randomized variant of the multiply-shift method works well on the average, when the program begins by picking  $a$  as a randomly chosen odd integer.

### 11.3.2 Random hashing

Suppose that a malicious adversary chooses the keys to be hashed by some fixed hash function. Then the adversary can choose  $n$  keys that all hash to the same slot, yielding an average retrieval time of  $\Theta(n)$ . Any static hash function is vulnerable to such terrible worst-case behavior. The only effective way to improve the situation is to choose the hash function *randomly* in a way that is *independent* of the keys that are actually going to be stored. This approach is called **random hashing**. A special case of this approach, called **universal hashing**, can yield provably good performance on average when collisions are handled by chaining, no matter which keys the adversary chooses.

To use random hashing, at the beginning of program execution you select the hash function at random from a suitable family of functions. As in the case of quicksort, randomization guarantees that no single input always evokes worst-case behavior. Because you randomly select the hash function, the algorithm can behave differently on each execution, even for the same set of keys to be hashed, guaranteeing good average-case performance.

Let  $\mathcal{H}$  be a finite family of hash functions that map a given universe  $U$  of keys into the range  $\{0, 1, \dots, m-1\}$ . Such a family is said to be **universal** if for each pair of distinct keys  $k_1, k_2 \in U$ , the number of hash functions  $h \in \mathcal{H}$  for which  $h(k_1) = h(k_2)$  is at most  $|\mathcal{H}|/m$ . In other words, with a hash function randomly chosen from  $\mathcal{H}$ , the chance of a collision between distinct keys  $k_1$  and  $k_2$  is no more than the chance  $1/m$  of a collision if  $h(k_1)$  and  $h(k_2)$  were randomly and independently chosen from the set  $\{0, 1, \dots, m-1\}$ .

Independent uniform hashing is the same as picking a hash function uniformly at random from a family of  $m^n$  hash functions, each member of that family mapping the  $n$  keys to the  $m$  hash values in a different way.

Every independent uniform random family of hash function is universal, but the converse need not be true: consider the case where  $U = \{0, 1, \dots, m-1\}$  and the only hash function in the family is the identity function. The probability that two distinct keys collide is zero, even though each key is hashed to a fixed value.



The following corollary to Theorem 11.2 on page 279 says that universal hashing provides the desired payoff: it becomes impossible for an adversary to pick a sequence of operations that forces the worst-case running time.

### Corollary 11.3

Using universal hashing and collision resolution by chaining in an initially empty table with  $m$  slots, it takes  $\Theta(s)$  expected time to handle any sequence of  $s$  INSERT, SEARCH, and DELETE operations containing  $n = O(m)$  INSERT operations.

**Proof** The INSERT and DELETE operations take constant time. Since the number  $n$  of insertions is  $O(m)$ , we have that  $\alpha = O(1)$ . Furthermore, the expected time for each SEARCH operation is  $O(1)$ , which can be seen by examining the proof of Theorem 11.2. That analysis depends only on collision probabilities, which are  $1/m$  for any pair  $k_1, k_2$  of keys by the choice of an independent uniform hash function in that theorem. Using a universal family of hash functions here instead of using independent uniform hashing changes the probability of collision from  $1/m$  to at most  $1/m$ . By linearity of expectation, therefore, the expected time for the entire sequence of  $s$  operations is  $O(s)$ . Since each operation takes  $\Omega(1)$  time, the  $\Theta(s)$  bound follows. ■

### 11.3.3 Achievable properties of random hashing

There is a rich literature on the properties a family  $\mathcal{H}$  of hash functions can have, and how they relate to the efficiency of hashing. We summarize a few of the most interesting ones here.

Let  $\mathcal{H}$  be a family of hash functions, each with domain  $U$  and range  $\{0, 1, \dots, m-1\}$ , and let  $h$  be any hash function that is picked uniformly at random from  $\mathcal{H}$ . The probabilities mentioned are probabilities over the picks of  $h$ .

- The family  $\mathcal{H}$  is **uniform** if for any key  $k$  in  $U$  and any slot  $q$  in the range  $\{0, 1, \dots, m-1\}$ , the probability that  $h(k) = q$  is  $1/m$ .
- The family  $\mathcal{H}$  is **universal** if for any distinct keys  $k_1$  and  $k_2$  in  $U$ , the probability that  $h(k_1) = h(k_2)$  is at most  $1/m$ .
- The family  $\mathcal{H}$  of hash functions is  **$\epsilon$ -universal** if for any distinct keys  $k_1$  and  $k_2$  in  $U$ , the probability that  $h(k_1) = h(k_2)$  is at most  $\epsilon$ . Therefore, a universal family of hash functions is also  $1/m$ -universal.<sup>2</sup>

---

<sup>2</sup> In the literature, a  $(c/m)$ -universal hash function is sometimes called  $c$ -universal or  $c$ -approximately universal. We'll stick with the notation  $(c/m)$ -universal.

- The family  $\mathcal{H}$  is ***d-independent*** if for any distinct keys  $k_1, k_2, \dots, k_d$  in  $U$  and any slots  $q_1, q_2, \dots, q_d$ , not necessarily distinct, in  $\{0, 1, \dots, m-1\}$  the probability that  $h(k_i) = q_i$  for  $i = 1, 2, \dots, d$  is  $1/m^d$ .

Universal hash-function families are of particular interest, as they are the simplest type supporting provably efficient hash-table operations for any input data set. Many other interesting and desirable properties, such as those noted above, are also possible and allow for efficient specialized hash-table operations.

### 11.3.4 Designing a universal family of hash functions

This section presents two ways to design a universal (or  $\epsilon$ -universal) family of hash functions: one based on number theory and another based on a randomized variant of the multiply-shift method presented in Section 11.3.1. The first method is a bit easier to prove universal, but the second method is newer and faster in practice.

#### A universal family of hash functions based on number theory

We can design a universal family of hash functions using a little number theory. You may wish to refer to Chapter 31 if you are unfamiliar with basic concepts in number theory.

Begin by choosing a prime number  $p$  large enough so that every possible key  $k$  lies in the range 0 to  $p-1$ , inclusive. We assume here that  $p$  has a “reasonable” length. (See Section 11.3.5 for a discussion of methods for handling long input keys, such as variable-length strings.) Let  $\mathbb{Z}_p$  denote the set  $\{0, 1, \dots, p-1\}$ , and let  $\mathbb{Z}_p^*$  denote the set  $\{1, 2, \dots, p-1\}$ . Since  $p$  is prime, we can solve equations modulo  $p$  with the methods given in Chapter 31. Because the size of the universe of keys is greater than the number of slots in the hash table (otherwise, just use direct addressing), we have  $p > m$ .

Given any  $a \in \mathbb{Z}_p^*$  and any  $b \in \mathbb{Z}_p$ , define the hash function  $h_{ab}$  as a linear transformation followed by reductions modulo  $p$  and then modulo  $m$ :

$$h_{ab}(k) = ((ak + b) \bmod p) \bmod m. \quad (11.3)$$

For example, with  $p = 17$  and  $m = 6$ , we have

$$\begin{aligned} h_{3,4}(8) &= ((3 \cdot 8 + 4) \bmod 17) \bmod 6 \\ &= (28 \bmod 17) \bmod 6 \\ &= 11 \bmod 6 \\ &= 5. \end{aligned}$$

Given  $p$  and  $m$ , the family of all such hash functions is

$$\mathcal{H}_{pm} = \{h_{ab} : a \in \mathbb{Z}_p^* \text{ and } b \in \mathbb{Z}_p\}. \quad (11.4)$$